

SQL PROJECT

Product Categories & Sales Analysis

**Blinkit Quick-Commerce Database - Customers,
Orders, Products, Categories & Inventory**

Project Overview & Objectives

Objectives

- Study Blinkit's e-commerce data: customers, orders, products, categories and inventory
- Measure sales performance across products and product categories
- Identify top customers, top products and the main revenue drivers
- Analyse delivery status, payment preferences and delivery-time performance
- Evaluate stock health - stock received vs damaged stock, min/max levels
- Provide data-backed insights to guide inventory, marketing and operations

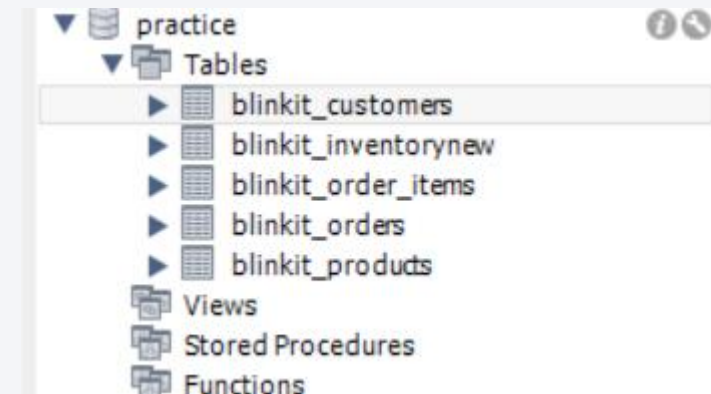
Scope & Dataset

5 core tables in the practice database:

- blinkit_customers - customer master data
- blinkit_orders - order header & fulfilment
- blinkit_order_items - line items (qty, price)
- blinkit_products - product & category master
- blinkit_inventorynew - stock received / damaged

Analysis layers:

- 1) Customer analysis
- 2) Sales analysis
- 3) Delivery & payment
- 4) Inventory analysis



Database Schema - Tables & Key Columns

Tables & Columns

blinkit_customers
customer_id, customer_name, address, area,
customer_segment, registration_date, total_orders

blinkit_orders
order_id, customer_id, order_date,
delivery_status, payment_method,
promised_delivery_time

blinkit_order_items
order_id, product_id, quantity, unit_price

blinkit_products
product_id, product_name, category,
min_stock_level, max_stock_level

blinkit_inventorynew
product_id, stock_received, damaged_stock

Relationships (JOIN chain)

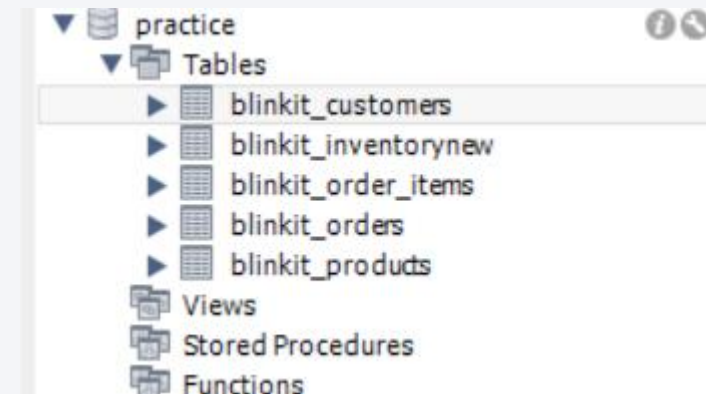
```
customers.customer_id <-- orders.customer_id
orders.order_id <-- order_items.order_id
products.product_id <-- order_items.product_id
products.product_id <-- inventorynew.product_id
```

Main JOIN path for sales:

customers --> orders --> order_items --> products

Stock analysis path:

products --> inventorynew



Methodology & SQL Techniques

Approach

- RDBMS: MySQL

Step 1 - Explore: browse tables and sample columns

Step 2 - Aggregate: totals, counts, sums

Step 3 - Join: combine all 5 tables on keys

Step 4 - Rank: Top-N via DENSE_RANK

Step 5 - Trend: DATE_FORMAT for time grouping

Step 6 - Insight: convert results into business findings

SQL Techniques Used

```
SELECT col1, col2 FROM table;
SELECT DISTINCT col FROM table;

FROM a JOIN b ON a.k = b.k    -- inner join

GROUP BY col  COUNT(*)  SUM(x*y)  ROUND(..,2)

DATE_FORMAT(date,'%Y')      -- year
DATE_FORMAT(date,'%M/%y')   -- month/year

DENSE_RANK() OVER (ORDER BY x DESC)  -- Top-N

WITH temp AS (...) SELECT ... FROM temp;
subquery: WHERE col IN (SELECT ...)

TIMESTAMPDIFF(MINUTE, d1, d2)  -- time gap
```

Dataset Snapshot - Selected Columns

Key Insights

- Data explored first with focused column selection
- Distinct product names listed with GROUP BY
- Distinct categories extracted for the category analysis
- These queries confirm the shape of the data before deep-dive

SQL Query

```
-- Selected columns
SELECT customer_name, address, registration_date
FROM blinkit_customers;

-- Distinct products
SELECT product_name FROM blinkit_products
GROUP BY product_name;

-- Categories
SELECT category FROM blinkit_products
GROUP BY category;
```

Output: Dataset Snapshot - Selected Columns

Result 1: customer_name | address | registration_date Result 2: distinct product_name list Result 3: distinct category list

customer_name	address	registration_date
Indrajit Pau	66, Vora Chowk Maheshtala 376030	2024-04-10
Ekiya Pau	H.No. 336 Lad Zila, Hajipur-409871	2023-11-29
Onkar Suresh	88/40, Bora Zila, Phusro-161515	2024-07-27
Jeet Gandhi	95/861, Jaggi, Sikar-343285	2024-09-19
Zansi Parsa	H.No. 40, Gopal Ganj, Bidar 854350	2023-08-01
Idika Basu	67, Karan Chowk Katihar 496332	2024-07-16

product_name	category
Baby Wipes	Baby Care
Baby Food	Personal Care
Toothpaste	Fruits & Vegetables
Potatoes	Dairy & Breakfast
Spinach	Household Care
Cheese	Pharmacy
Dish Soap	Instant & Frozen Food

Key Business Metrics - Overall Totals

Key Insights

- Counts give the total scale: customers, orders and products
- SUM(stock_received) shows total inbound stock
- SUM(damaged_stock) shows total stock wastage
- Baseline figures used throughout the analysis

SQL Query

```
-- Total customers
SELECT COUNT(customer_name) FROM blinkit_customers;

-- Total orders
SELECT COUNT(order_id) FROM blinkit_orders;

-- Total products
SELECT COUNT(DISTINCT product_name) AS products
FROM blinkit_products;

-- Total stock received / damaged
SELECT SUM(stock_received) FROM blinkit_inventorynew;
SELECT SUM(damaged_stock) FROM blinkit_inventorynew;
```

Output: Key Business Metrics - Overall Totals

Results (single values): total customers, total orders, total products, total stock_received, total damaged_stock

	count(customer_name)
▶	2500

	count(order_id)
▶	1061

	sum(stock_received)
▶	28779

	sum(damaged_stock)
▶	1109

Customer Growth by Year

Key Insights

- DATE_FORMAT extracts the registration year
- Trend of new customers year-over-year
- Shows growth phases and slowdown periods
- Basis for acquisition and retention planning

SQL Query

```
SELECT DATE_FORMAT(registration_date,'%Y') AS years,
       COUNT(*) AS counts
FROM blinkit_customers
GROUP BY years;
```

Output: Customer Growth by Year

Result: years | counts (number of customers registered that year)

	years	counts
▶	2024	1337
	2023	1163

Area-wise Customer Distribution

Key Insights

- Customer count grouped by delivery area
- High-density areas are priority delivery zones
- Helps plan supply-chain coverage and marketing spend
- Low-density areas may need campaigns or expansion

SQL Query

```
SELECT area, COUNT(customer_name) AS count_by_area
FROM blinkit_customers
GROUP BY area;
```

Output: Area-wise Customer Distribution

Result: area | count_by_area (customers per area)

	area	count_by_area
▶	Sultan Pur Majra	7
	Chinsurah	7
	Kurnool	3
	Korba	13
	Kavali	7
	Madanapalle	5

Customer Segment Analysis

Key Insights

- Total orders aggregated by customer_segment
- Identifies the highest-order-value segments
- Also analysed per-customer segment with order totals
- Guides targeted offers for valuable segments

SQL Query

```
-- Segment-wise total orders
SELECT customer_segment, SUM(total_orders) AS total_orders
FROM blinkit_customers
GROUP BY customer_segment;

-- Each customer with their segment & orders
SELECT customer_name, customer_segment,
       SUM(total_orders) AS total_orders_placed
FROM blinkit_customers
GROUP BY customer_segment, customer_name;
```

Output: Customer Segment Analysis

Result 1: customer_segment | total_orders Result 2: customer_name | customer_segment | total_orders_placed

	customer_segment	total_orders
▶	Regular	6598
	New	6829
	Inactive	6350
	Premium	6452

	customer_name	customer_segment	total_orders_placed
▶	Indrajit Pau	Regular	18
	Ekiya Pau	New	18
	Onkar Suresh	New	19
	Jeet Gandhi	Regular	13
	Zansi Parsa	Inactive	20
	Idika Basu	Inactive	12

Month & Year-wise Sales Trend

Key Insights

- Revenue summed per month/year across 3-table JOIN
- Reveals seasonal peaks and slow months
- ROUND keeps revenue figures readable
- Input for demand planning and promo timing

SQL Query

```
SELECT DATE_FORMAT(registration_date,'%M/%y') AS month_year,  
       ROUND(SUM(oi.quantity * oi.unit_price)) AS totals  
FROM blinkit_customers c  
JOIN blinkit_orders o    ON c.customer_id = o.customer_id  
JOIN blinkit_order_items oi ON oi.order_id = o.order_id  
GROUP BY month_year;
```

Output: Month & Year-wise Sales Trend

Result: month_year | totals (sales value per month/year)

	month_year	totals
▶	September/24	58172
	November/23	48259
	February/24	62505
	January/24	50091
	October/23	41808
	April/24	47465

Customer x Product Sales Matrix

Key Insights

- 4-table JOIN builds a customer-product sales matrix
- Shows which customers buy which products
- Highlights cross-sell and personalisation potential
- $\text{SUM}(\text{qty} * \text{unit_price}) = \text{revenue per combination}$

SQL Query

```
SELECT c.customer_name, p.product_name,  
       SUM(oi.quantity * oi.unit_price) AS total  
FROM blinkit_orders o  
JOIN blinkit_customers c ON c.customer_id = o.customer_id  
JOIN blinkit_order_items oi ON oi.order_id = o.order_id  
JOIN blinkit_products p ON p.product_id = oi.product_id  
GROUP BY c.customer_name, p.product_name;
```

Output: Customer x Product Sales Matrix

Result: customer_name | product_name | total (spend per customer-product pair)

	customer_name	product_name	total
▶	Xiti Sachdeva	Baby Wipes	169.88
	Avni Mistry	Baby Wipes	339.76
	Matthew Bhatti	Baby Wipes	509.64
	Darika Sharaf	Baby Wipes	339.76
	Xalak Sangha	Baby Wipes	169.88
	Chameli Grover	Baby Wipes	169.88

Top 10 Customers by Revenue

Key Insights

- DENSE_RANK window function ranks customers by spend
- CTE (WITH temp) keeps the ranking query clean
- Top 10 customers = high-value base for loyalty
- Recommendation: targeted retention for this group

SQL Query

```
WITH temp AS (  
    SELECT customer_name,  
           ROUND(SUM(oi.quantity * oi.unit_price)) AS total  
    FROM blinkit_orders o  
    JOIN blinkit_order_items oi ON oi.order_id = o.order_id  
    JOIN blinkit_customers c   ON c.customer_id = o.customer_id  
    GROUP BY customer_name)  
SELECT *, DENSE_RANK() OVER (ORDER BY total DESC) AS ranks  
FROM temp  
WHERE ranks <= 10;
```

Output: Top 10 Customers by Revenue

Result: customer_name | total | ranks (filtered to ranks <= 10)

	customer_name	total	ranks
►	Eta Srinivasan	7686	1
	Maanav Gupta	6499	2
	Upma Salvi	5197	3
	Leena Deol	5147	4
	Azad Sachdev	5130	5
	Leena Loyal	4935	6

Top 10 Products by Revenue

Key Insights

- Revenue computed per product via JOIN
- Chained CTEs + DENSE_RANK give the Top-10
- Best-selling products are restocking priorities
- Feeds assortment and inventory decisions

SQL Query

```
WITH temp AS (  
    SELECT p.product_name,  
           ROUND(SUM(oi.quantity * oi.unit_price)) AS total  
    FROM blinkit_order_items oi  
    JOIN blinkit_products p ON p.product_id = oi.product_id  
    GROUP BY p.product_name),  
temp2 AS (  
    SELECT product_name, total,  
           DENSE_RANK() OVER (ORDER BY total DESC) AS ranks  
    FROM temp)  
SELECT * FROM temp2 WHERE ranks <= 10;
```

Output: Top 10 Products by Revenue

Result: product_name | total | ranks (top 10 products)

	product_name	total	ranks
▶	Vitamins	69931	1
	Cough Syrup	40393	2
	Baby Wipes	38582	3
	Butter	36956	4
	Pet Treats	36663	5
	Pain Reliever	35009	6

Top 10 Categories by Revenue

Key Insights

- Category revenue computed with GROUP BY + JOIN
- DENSE_RANK used inside a subquery for Top-10
- Leading categories guide assortment & promotions
- Directly addresses the project title - category analysis

SQL Query

```
SELECT * FROM (  
  SELECT p.category,  
         ROUND(SUM(oi.quantity * oi.unit_price)) AS total,  
         DENSE_RANK() OVER  
           (ORDER BY SUM(oi.quantity * oi.unit_price) DESC) AS ranks  
  FROM blinkit_order_items oi  
  JOIN blinkit_products p ON p.product_id = oi.product_id  
  GROUP BY p.category) t  
WHERE ranks <= 10;
```

Output: Top 10 Categories by Revenue

Result: category | total | ranks (top 10 categories)

	category	total	ranks
►	Dairy & Breakfast	146792	1
	Pharmacy	145334	2
	Fruits & Vegetables	130903	3
	Pet Care	96032	4
	Cold Drinks & Juices	85835	5
	Snacks & Munchies	84488	6

Delivery Status Analysis

Key Insights

- Order count and sales value per delivery_status
- Share of delivered vs delayed / cancelled orders
- Sales value lost on non-delivered orders
- Operational quality metric for quick-commerce SLA

SQL Query

```
SELECT o.delivery_status,  
       COUNT(*) AS count,  
       ROUND(SUM(oi.quantity * oi.unit_price),2) AS total  
FROM blinkit_orders o  
JOIN blinkit_order_items oi ON o.order_id = oi.order_id  
GROUP BY delivery_status;
```

Output: Delivery Status Analysis

Result: delivery_status | count | total (sales value per status)

	delivery_status	count	total
▶	On Time	741	710471.14
	Slightly Delayed	222	230824.96
	Significantly Delayed	98	101722.12

Payment Method Analysis

Key Insights

- Order count and sales value per payment_method
- Shows most-used payment modes and their value
- Guides payment-promo and COD risk strategy
- Also output: customer-wise status & payment view

SQL Query

```
SELECT o.payment_method,  
       COUNT(*) AS count,  
       ROUND(SUM(oi.quantity * oi.unit_price),2) AS total  
FROM blinkit_orders o  
JOIN blinkit_order_items oi ON o.order_id = oi.order_id  
GROUP BY payment_method;
```

Output: Payment Method Analysis

Result: payment_method | count | total (sales value per method)

	payment_method	count	total
▶	UPI	266	263452.25
	Wallet	247	230700.57
	Card	268	273729.52
	Cash	280	275135.88

Delivery Time Analysis

Key Insights

- TIMESTAMPDIFF measures gap in minutes
- order_date vs promised_delivery_time difference
- Reveals how tightly delivery promises are kept
- Key SLA metric for a quick-commerce platform

SQL Query

```
SELECT customer_id, order_date,
       promised_delivery_time,
       TIMESTAMPDIFF(MINUTE, order_date,
                    promised_delivery_time) AS Minute_difference
FROM blinkit_orders;
```

Output: Delivery Time Analysis

Result: customer_id | order_date | promised_delivery_time | Minute_difference

	customer_id	order_date	promised_delivery_time	Minute_difference
	15808945	2024-10-23 05:23:29	2024-10-23 05:34:29	11
	48281892	2023-04-02 03:45:11	2023-04-02 04:00:11	15
▶	89617089	2024-05-23 03:21:47	2024-05-23 03:37:47	16
	44174426	2023-06-09 12:10:20	2023-06-09 12:25:20	15
	51476157	2023-04-30 20:52:21	2023-04-30 21:11:21	19
	93127511	2023-10-20 10:54:27	2023-10-20 11:07:27	13

Inventory - Stock Received & Damaged

Key Insights

- Stock inflow (received) summed per product
- Damaged stock exposes wastage hotspots
- High damaged ratio products need quality review
- Feeds reorder quantity decisions

SQL Query

```
SELECT p.product_name, SUM(s.stock_received) AS stock_recived
FROM blinkit_products p
JOIN blinkit_inventorynew s ON p.product_id = s.product_id
GROUP BY p.product_name;
```

```
SELECT p.product_name, SUM(s.damaged_stock) AS damaged_stock
FROM blinkit_products p
JOIN blinkit_inventorynew s ON p.product_id = s.product_id
GROUP BY p.product_name;
```

Output: Inventory - Stock Received & Damaged

Result 1: product_name | stock_recived Result 2: product_name | damaged_stock

	product_name	stock_recived
▶	Baby Wipes	695
	Baby Food	448
	Toothpaste	464
	Potatoes	800
	Spinach	117
	Cheese	535

	product_name	damaged_stock
▶	Baby Wipes	24
	Baby Food	16
	Toothpaste	18
	Potatoes	31
	Spinach	4
	Cheese	20

Stock Levels - Min / Max & Largest Receipt

Key Insights

- min_stock_level and max_stock_level tracked per product
- Subquery finds the product with the largest stock receipt
- Flags items near their reorder threshold
- Supports automated replenishment planning

SQL Query

```
SELECT product_name, MIN(min_stock_level) AS min_stock_level
FROM blinkit_products
GROUP BY product_name;

SELECT product_name, MAX(max_stock_level) AS max_stock_level
FROM blinkit_products
GROUP BY product_name;

-- Product with max stock received
SELECT product_name FROM blinkit_products
WHERE product_id IN (
  SELECT product_id FROM blinkit_inventorynew
  WHERE stock_received =
    (SELECT MAX(stock_received)
     FROM blinkit_inventorynew));
```

Output: Stock Levels - Min / Max & Largest Receipt

Results: product_name | min_stock_level | product_name | max_stock_level | product name with the max stock_received

	product_name	min_stock_level
►	Baby Wipes	15
	Baby Food	11
	Toothpaste	18
	Potatoes	11
	Spinach	14
	Cheese	12

	product_name	max_stock_level
►	Baby Wipes	100
	Baby Food	86
	Toothpaste	85
	Potatoes	94
	Spinach	53
	Cheese	95

	product_name
►	Baby Food
	Potatoes
	Spinach
	Cheese
	Dish Soap
	Vitamins

Key Findings & Conclusion

Key Insights

- Revenue is concentrated in a few top customers, products and categories
 - focus effort there
- Clear monthly sales pattern supports seasonal stock & promo planning
- Delivery status and payment-method analysis reveal operational strengths and risks
- Inventory gaps between received vs damaged stock and min/max levels flag replenishment needs
- DENSE_RANK + CTEs + multi-table JOINS were the core SQL techniques applied

Conclusion

The SQL analysis turns raw quick-commerce data into actionable insights on customers, sales, delivery performance and inventory health, supporting smarter business decisions for Blinkit.

Future Scope

Predictive demand forecasting, customer churn analysis, cohort analysis, and real-time dashboards built on the same data model.